

School of Computer Science

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES



UNIVERSITY OF LEEDS

Final Report

Using Non-Photorealistic Rendering Techniques to Create Efficient Real-Time Stylised Oceans

Jacob Onion

**Submitted in accordance with the requirements for the degree of
BSc Computer Science**

2025/26

COMP3931 Individual Project

The candidate confirms that the following have been submitted:

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (29/04/26)
Link to online code repository	URL	Sent to supervisor and assessor (29/04/26)

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of student) Jacob Onion

Summary

This project aims to create a stylised water system that uses a combination of water simulation algorithms and NPR techniques. The result is an ocean simulation that emulates 2D animation in a 3D environment and can be used in real-time contexts. The component-based structure of the project allows for high levels of customization in the water aesthetics.

Acknowledgements

I would like to thank my supervisor Dr Rafael Kuffner Dos Anjos and my assessor Dr Hamish Carr for providing me with advice throughout this project.

I acknowledge the use of Claude AI to assist with initial research related to my project, planning the structure of my report, checking grammar, and debugging code. All technical content, design decisions, implementation, and analysis are my own work. Any AI-generated outputs were critically reviewed, verified, and rewritten in my own words before inclusion.

Table of Contents

Summary.....	iii
Acknowledgements.....	iv
Table of Contents	v
Chapter 1 Introduction and Background Research.....	1
1.1 Introduction	1
1.2 System Requirements.....	1
1.3 Background Research.....	1
1.3.1 Wave simulation algorithms	1
1.3.1.1 Parametric models	1
1.3.1.2 Spectral models	2
1.3.1.3 Physics-based methods	3
1.3.2 Water surface rendering.....	3
1.3.2.1 Lighting.....	3
1.3.2.2 Reflections	3
1.3.2.3 Foam rendering.....	5
1.3.3 Dynamic interactions	5
1.3.3.1 Grid-based simulation	5
1.3.3.2 Analytical methods	6
1.3.3.3 Splash generation	6
1.3.4 Non-Photorealistic Rendering and Stylisation	6
1.3.4.1 Toon and cel shading.....	6
1.3.4.2 Colour quantisation	7
1.3.4.3 NPR as a performance benefit.....	7
Chapter 2 Methodology	8
2.1 Wave Simulation	8
2.1.1 Wave shaping function.....	8
2.1.2 Wave summation	9
2.1.3 Normal calculation.....	10
2.2 Dynamic Interactions.....	11
2.2.1 Ripple wavefront function	11
2.2.2 Normals calculation.....	12
2.2.3 Ripple management	13
2.2.4 Splash particles.....	13

2.3 Foam rendering	13
2.4 Reflections	14
2.4.1 Cubemap reflections	14
2.4.2 Screen-Space Reflections	15
2.4.2.1 Ray computation	15
2.4.2.2 Ray marching	15
2.4.2.3 Hit detection	16
2.4.2.4 Binary search refinement	16
2.4.2.5 Boundary and validity checks:	17
2.4.3 Fresnel blending	17
2.4.4 Distance Fade	18
2.5 Organisation	18
2.5.1 Gantt Chart	18
2.5.2 Version control	19
Chapter 3 Implementation and Validation	20
3.1 Chosen tools	20
3.2 Software Architecture	20
3.2.1 Wave simulation	21
3.2.2 Vertex stage	21
3.2.3 CPU – GPU Communication	21
3.2.4 SSR renderer feature	22
3.2.5 Fragment stage	22
3.3 Software validation	23
3.4 GUI	24
3.4.1 Shader Parameters	24
Chapter 4 Results, Evaluation and Discussion	25
4.1 Evaluation	25
4.1.1 Performance	25
4.1.1.1 Wave Count	25
4.1.1.2 SSR step count	26
4.1.1.3 Ripple Count	26
4.1.2 Visual Analysis	27
4.2 Conclusions	29
4.3 Future work	30

List of References	31
Appendix A Self-appraisal	33
A.1 Critical self-evaluation	33
A.2 Personal reflection and lessons learned	34
A.3 Legal, social, ethical and professional issues	34
A.3.1 Legal issues	34
A.3.2 Social issues	35
A.3.3 Ethical issues	35
A.3.4 Professional issues	35
Appendix B External Materials	37

Chapter 1

Introduction and Background Research

1.1 Introduction

Traditionally, real-time rendering has been focused on achieving physical accuracy over stylised visuals that require deliberate abstraction and exaggeration. Most ocean simulation research is on techniques that are too computationally expensive to be used in a real-time context and lack the customisability for artists to achieve a desired style. This project investigates how various cheaper rendering techniques can be employed to achieve stylised water that runs in real-time and reacts dynamically to its environment.

1.2 System Requirements

The requirements for this project to be successful are as follows:

1. The simulation must produce convincing ocean surface motion.
2. The system must support real-time dynamic interactions with scene geometry that integrate visually with the underlying simulation.
3. The output must achieve a coherent non-photorealistic visual style drawing on the conventions of 2D animation and prior stylised real-time water.
4. The simulation must be performant enough to be used in real-time contexts on a variety of machines.

1.3 Background Research

1.3.1 Wave simulation algorithms

There are many approaches to simulating wave motion, ranging from complete fluid simulations to lightweight parametric models. Each option represents a different point on the spectrum between physical accuracy and computational cost. Darles et al. provides a comprehensive overview of the commonly used methods, categorised into parametric, spectral and physical models [1].

1.3.1.1 Parametric models

Parametric models represent the ocean surface as a sum of periodic wave functions, defined by a set of parameters representing variables such as amplitude and wavelength. By layering many of these waves with varying parameters, complex and visually convincing ocean surfaces can be created [2]. This approach is commonly referred to as the sum-of-

sines, and was formally established in real-time graphics by Finch, who demonstrated its use in a vertex shader to produce animated ocean geometry [3].

This model is used with a variety of periodic functions for different wave shapes, the most common being the Gerstner function [4]. Rather than applying displacement to vertices vertically only, Gerstner waves apply an additional horizontal displacement, creating the peaks seen in steep waves. However, these waves can collapse in on themselves and deform if not tuned finely, therefore other functions may be more suitable depending on the use case.

The primary advantage of parametric models is their low computational cost and how parallelisable they are: each vertex position is calculated independently from one another with no dependencies. This makes them well suited to real-time applications where performance is a concern. The main issue with these models is that the resulting waves are deterministic and will repeat over large distances. However, Fractional Brownian motion, constructed by layering waves of increasing frequencies and decreasing amplitudes, is often merged with these models to break up the repetition [23]. Domain warping (using the output of one wave as part of the input for the next wave) is also used to enhance the turbulence of the surface [5].

1.3.1.2 Spectral models

Spectral models take a different approach, using a frequency-domain representation for the ocean surface. We can see an example of this being used by Tessendorf, who models the surface of the ocean using the Phillips spectrum (a description of the wave energy distribution produced by observing oceans) [6]. An initial frequency-domain height field is generated based on this, then converted to the spatial domain using an inverse Fast Fourier Transform (FFT) so it can be used to animate the ocean surface.

This FFT approach produces more physically accurate ocean surfaces than parametric models and is used commonly in the film and games industry. For example, Ang et al. discusses how *Sea of Thieves* used the approach discussed by Tessendorf for their ocean simulation [7, 6]. The primary limitation of spectral models in real-time contexts is the computational cost: the FFT must be executed each frame, which is demanding in large oceans. This method is also more restrictive on artistic control than parametric models since wave characteristics are produced from the spectrum rather than being specified explicitly.



Figure 1.1: Screenshot of the FFT-based waves in Sea of Thieves [7]

1.3.1.3 Physics-based methods

At the far end of the computational cost spectrum, physics-based methods simulate water as a fluid controlled by the Navier-Stokes equations, accounting for pressure, viscosity and incompressibility. These methods can produce highly realistic behaviour such as breaking waves and splashes, but the computational cost is currently unrealistic for real-time use [1].

1.3.2 Water surface rendering

1.3.2.1 Lighting

The optical behaviour of water surfaces is determined largely by the Fresnel effect, which describes how the amount of light reflected on the surface varies with the angle of incidence. At shallow angles, water appears highly reflective, while at higher angles it becomes more transparent. This relationship is generally approximated using the Schlick approximation which provides a cheap heuristic for real-time contexts [8].

Water surfaces also often have sharp highlights when illuminated by a direct light source. These are generally computed using Blinn-Phong or BRDF and are also influenced by the Fresnel effect.

1.3.2.2 Reflections

Water surfaces are highly reflective, so reflections make up a large amount of their visual identity. Several approaches exist in real-time rendering, each with their own benefits and trade-offs.

Cubemap reflections represent the surrounding environment as a cube with each face representing a direction on the major axes. Given the viewing direction and surface normal at each point on the water surface the reflected vector is calculated using $r = 2(n * v)n - v$. This vector is then used to sample the cubemap and map the contents onto the water.

Cubemap reflections remain widely used for sky and environmental reflections due to their cost and simplicity [9]. However, their fundamental limitation is that they utilise a single snapshot from a single point, meaning they are incapable of reflecting dynamic objects or multiple objects lined up on the same vector from the origin.

Environment probes extend the cubemap concept by placing multiple capture points throughout the scene, with reflections blending between them based on world position. This mitigates the single-viewpoint limitation, and is used in modern game engines for local reflections [10]. However, the reflections are still unable to capture dynamic objects, and blending between probes can introduce its own artifacts.

Planar reflections take a different approach by rendering the scene through a second camera position reflected about the water surface and writing the output to a render texture which is sampled by the water shader. Since the reflection is an additional scene render it is completely accurate. It solves the dynamic objects limitation of reflection mapping techniques, and has none of the missing-geometry artifacts produced by screen space techniques. The primary drawback is cost; an additional scene render per reflective surface makes it impractical for complex scenes. It is also only physically accurate for a flat surface. Since the reflection was captured from a planar camera any displacement on the plane such as waves introduces inaccuracies. For these reasons, planar reflections are generally reserved for small, enclosed surfaces rather than large open environments where the limitations are hard to justify.

Screen Space Reflections (SSR) follow another unique approach by ray marching through the depth buffer in screen space searching for collisions with scene geometry. When a ray intersects with the buffer the screen is sampled at that point for the reflection. Since the depth buffer represents the current frame, SSR produces reflections that can capture dynamic objects and shift correctly with camera movement [11]. Since the cost is proportional to the marching depth and resolution rather than the scene complexity it is much more predictable than other techniques. However, SSR has well-defined failure cases: anything not currently on the screen cannot be reflected so objects off-camera and complex geometry can produce missing or cut-off reflections.

Hardware ray tracing is a technique that is gaining more popularity in recent years, where rays are traced through the world space geometry, testing for intersections to return as surface hits. These reflections are fully accurate and are free from the limitations of the previous techniques [12]. However, ray tracing in real time requires dedicated hardware that is only available in modern GPUs. The computational cost also scales with the scene complexity due to it running in world space, leading to unpredictable performance.

A common strategy used in titles such as Sea of Thieves is to layer these approaches to mitigate their weaknesses [7]. For example, a cubemap or environment probe handles

distant reflections cheaply, while SSR is applied on top for local geometry. The environment probe reflections are blended with SSR based the reflection quality to reduce screen-space artifacts. This mitigates the core weaknesses of SSR without the cost of more expensive techniques such as ray tracing.

1.3.2.3 Foam rendering

Foam appears on ocean surfaces when the water is turbulent, for example at wave crests and wakes. In real-time rendering, foam is generally represented as a texture blended onto the water surface with the blend weight determined by the wave intensity at each point.

The simplest way of rendering foam is by deriving it directly from wave crest height. Any part of the surface above a defined height threshold are considered to be a wave crest, and the foam opacity is determined by how far above the threshold that fragment is. This approach lacks the physical accuracy of other methods but is computationally trivial. It is also highly versatile as it is decoupled from the wave simulation and gives the artist direct control over where foam appears. Finch uses this approach in his GPU Gems water implementation, blending a foam texture based on the wave height [3].

A more physically accurate approach introduced by Tessendorf derives foam coverage from the Jacobian determinant of the horizontal wave displacement [13]. As wave steepness increases, these displacements can cause the surface to fold back on itself, which is measured by the Jacobian becoming negative. This threshold corresponds with the physical act of waves breaking in real fluids. The Jacobian is computed per fragment and used to mask a foam texture where breaking is most likely [14]. The approach is cheap and requires no additional render passes, making it well suited to real-time contexts. However, it only captures the moment of breaking and does not measure foam persistence, meaning foam will appear intermittently rather than drifting as it would physically.

1.3.3 Dynamic interactions

Since fluids are highly physically responsive, the ripples and splashes that result from collisions are vital for communicating this property to the viewer. Techniques on real-time water interaction are broadly divided into two approaches: grid-based methods that propagate disturbances across the surface and parametric methods that evaluate each disturbance independently.

1.3.3.1 Grid-based simulation

The grid-based techniques for real-time interactive water surfaces were pioneered by Kass and Miller, who defined the water surface as a height field where each cell holds a scalar height value [15]. Disturbances propagate by updating each cell based on its neighbours

each frame, leading to emergent behaviour: ripples spread outwards and superimpose with each other without any individual wave events.

1.3.3.2 Analytical methods

Analytical approaches represent each disturbance as a separate parametric function evaluated at each vertex. A ripple event is defined by its origin, start time, and shape parameters, and its height is computed from a set expression using these values. Since the function has no dependency on neighbouring vertices, it is trivially parallelisable on the GPU.

Yuksel, House and Keyser formalised this with Wave Particles where discrete disturbances carry expanding wavefronts that are layered onto a height field [17]. The method is unconditionally stable and achieves high frame rates (170 fps in complex scenes) but because the particles spread independently it does not exhibit true wave to wave interaction such as diffraction.

The main trade-off of these techniques is physical accuracy versus computational cost and predictability. Grid methods produce realistic interference with other waves and objects, but at a cost that scales with grid resolution, making it difficult to utilise for large bodies of water. Analytic methods are perfectly predictable, cost only as much as the number of active disturbances, and don't require a persistent state, but each disturbance propagates in isolation from the others.

1.3.3.3 Splash generation

Splashes in real-time graphics are generally rendered as billboard particles subject to gravity. O'Brien and Hodgins introduced a version of this where the particles are emitted whenever the local upward velocity exceeds a threshold which remains widely used in real-time applications [18].

1.3.4 Non-Photorealistic Rendering and Stylisation

Non-photorealistic rendering (NPR) describes techniques that deliberately depart from physical accuracy to create visuals in a more artistic style. Kyprianidis et al. provides a comprehensive summary of these techniques, classifying them across abstraction, painterly rendering and cartoon-style shading [19]. This survey is used here as the structure for the fields discussed below.

1.3.4.1 Toon and cel shading

The main paradigm for cartoon-style rendering is toon shading, where diffuse lighting is quantised into discrete bands to replicate a hand-drawn animation look. Lake et al. formalised the real-time implementation of this technique, introducing a method that takes the angle between a surface and a light source and uses it to look up a pre-defined set of flat

colour bands [20]. They also used a technique for drawing outlines around 3D objects by rendering a second, slightly enlarged version of the object so only its outer edges are visible to the camera. Both techniques add negligible overhead to the standard lighting pass and so became the industry standard for stylised games rendering. Barla et al. extended the 1D texture used in Lake's method to utilise a 2D texture instead, allowing for richer effects such as ariel perspective and backlighting at approximately 20% additional cost to the original method [21] [20]. For edge detection Saito and Takahashi demonstrated that sudden changes in depth and normals could be used in outline detection, decoupling edge detection from scene geometry [22].

1.3.4.2 Colour quantisation

Colour reduction is a core NPR technique that maps continuous colours to a restricted palette of discrete tones, reinforcing the hand-drawn appearance of toon shading. In real-time shaders, this is typically performed using a posterization step: dividing and flooring colour channels to snap values to a discrete grid, achievable in a single fragment shader pass.

1.3.4.3 NPR as a performance benefit

NPR tends to result in better performance over physical rendering: by deliberately abstracting away from physical accuracy, expensive lighting models can be substituted for cheap approximations. For example, while a physically based renderer makes use of expensive techniques such as multi-bounce global illumination to produce images, a stylised equivalent can be produced using posterized shading bands and threshold-based highlights. This trade is viable because the later stylisation passes end up covering up the inaccuracies that would otherwise disrupt the simulation. In some cases, these inaccuracies are intentionally exaggerated as an artistic choice, such as a weaker version of the Fresnel effect to produce more vivid reflections. For lower-end hardware in particular this reframes stylisation not merely as an aesthetic decision but as a deliberate performance strategy.

Chapter 2 Methodology

2.1 Wave Simulation

The primary visual identity of any water rendering system is the motion of the surface. As established in section 1.3.1, wave simulations generally fall into two categories: spectral models and parametric models. This section describes and justifies the algorithm used in this project.

A parametric sum-of-sines model was selected over a spectral FFT approach. While FFT-based implementations produce more physically accurate surfaces, they introduce several issues that conflict with the goals of this project.

FFT requires generating a frequency-domain heightfield, performing an inverse transform, and storing the result in a texture every frame. This is far more computationally expensive than the per-vertex analytical function used in parametric models. On top of this, the improved visual fidelity of FFT approaches is mitigated in an NPR context, since the fine detail that is unobtainable in parametric models is masked by the stylisation passes performed later in the pipeline. A sum-of-sines model provides sufficient visual complexity for a stylised aesthetic, while being cheap and trivially parallelisable on the GPU.

Additionally, parametric models offer artistic control over waves via parameters, while FFT surfaces are shaped by a statistical spectrum that is harder to direct towards a visual target.

2.1.1 Wave shaping function

The standard sine function was replaced with a nonlinear parametric function to produce more physically plausible wave shapes:

$$s = e^{\sin(\varphi)} - 1$$

Taking the exponent of a sine naturally biases the waveform upwards at peaks and flattens it at troughs, resulting in sharper shapes that better represent real ocean movements.

Gerstner waves were considered for additional horizontal displacement but deemed unnecessary given the NPR nature of this project, where small geometric details are suppressed. This approach was adapted from Acerola's public implementation [24].

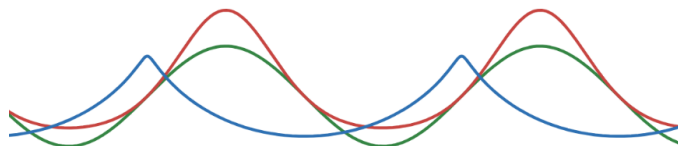


Figure 2.1: Comparison of sine (green), exponent of sine (red), and Gerstner waves (blue)

2.1.2 Wave summation

The water surface is computed as a summation of a set number of wave functions with varying parameters. Each wave i is defined by an amplitude A , a frequency w , and a direction d . The height contribution of the wave i at surface position p is:

$$h = A * (e^{\sin(\varphi)} - 1)$$

where the phase φ is given by:

$$\varphi = w * (d * p) + t$$

And t is the current simulation time. The total surface height at any point is the sum of all wave displacements:

$$H(p) = \sum_{i=0}^{N-1} h$$

This is the fundamental sum-of-sines structure, and there are three key techniques that can be added to produce a more natural and complex surface:

Procedural wave directions: instead of manually specifying the direction each wave should move in, directions are generated procedurally using a hash function that maps the wave index to a random vector. This ensures a naturally varied wave distribution as the wave count increases.

Domain warping: the phase of each wave incorporates the derivative of the preceding wave, creating inter-wave dependency. For a given wave i , the derivative $\delta\{i - 1\}$ from wave $i - 1$ is added to the spatial component of i 's phase before frequency scaling. This means the phase formula becomes:

$$\varphi = w * (d * p + \delta\{i - 1\}) + t$$

This is a form of domain warping, where the input of one function is distorted by the output of another [5]. When used in wave simulations, it breaks the repetition that results from summing independent functions, resulting in more organic waves that visually influence each other.

Fractional Brownian motion (fBm): After every wave iteration, the amplitude is scaled by a factor a , and the frequency is scaled by a correspond factor b :

$$A_{\{i+1\}} = A_i * a$$

$$w_{\{i+1\}} = w_i * b$$

Where $a < 1$ and $b > 1$. This is a variation on the fractional Brownian motion described by Mandelbrot and Van Ness, in which each iteration contributes to finer details at a reduced magnitude. The result is a surface with a few dominant waveforms creating the surface

shape, overlaid with progressively smaller ripples to produce the micro-movements found in real ocean water [25]. The number of waves can be adjusted to balance visual complexity against computational cost.

2.1.3 Normal calculation

Accurate normals are essential for all subsequent parts of the rendering process: lighting, reflections and toon-shading all depend on them. Precise normals can also be used in the lighting pass to add further detail to the surface without the need for more vertices. Two approaches exist for calculating normals on a procedurally displaced surface: finite differences and analytical differentiation.

Finite differences approximate the surface gradient by computing the height function at neighbouring points and computing the slope. While simple to implement, this approach requires multiple extra iterations of the wave function per vertex, increasing the computational cost proportionately. It also may not be precise depending on the sampling step size used: too large and fine details may be missed, too small and floating-point rounding errors arise.

The **analytical** approach finds normals by differentiating the height function directly with respect to the surface coordinates, deriving a closed form expression for the gradient. Since a parametric model with a predefined wave function was chosen, the partial derivative with respect to the XZ - plane can be computed exactly via the chain rule:

For a single wave with a shaping function $s_i = e^{\sin(\varphi_i)} - 1$, phase φ_i , and direction $d_i = (d_x, d_z)$, the partial derivatives with respect to the X and Z axis are:

$$\partial h_i / \partial x = A_i \cdot \omega_i \cdot d_x \cdot \cos(\varphi_i) \cdot e^{\sin(\varphi_i)}$$

$$\partial h_i / \partial z = A_i \cdot \omega_i \cdot d_z \cdot \cos(\varphi_i) \cdot e^{\sin(\varphi_i)}$$

These are accumulated across all waves and used to construct the surface normal:

$$n = \text{normalize}(-\partial H / \partial x, 1, -\partial H / \partial z)$$

This approach was selected because it is both cheaper and more accurate than the finite differences in this context. For a simulation with N waves, rather than requiring four additional evaluations of the wave function per vertex, 5N total, the analytical method requires a single cosine and multiply per wave.

2.2 Dynamic Interactions

Real-time water simulations must react to objects interacting with the surface. The background research identified three approaches to this problem: grid-based methods, wave particles, and analytic models.

Grid-based methods offer the greatest physical accuracy of these options, but require a dedicated simulation grid that must be updated each frame, introducing a memory overhead that scales with the grid resolution, quickly becoming a bottleneck on lower-end hardware. Wave particles don't require a grid, but need to manage complex logic along with a dynamically growing particle population. Neither approach is appropriate for this project's goal of targeting weaker hardware while maintaining consistent real-time performance.

This system instead adopts an analytic ripple model. Each interaction event generates a single ring wavefront evaluated directly in the vertex shader, with no grid required, no state maintained and no dependence on other vertices. This approach is similar to the wave particle concept from Yuksel, House and Keyser but simplified so each wavefront is represented as a single ring rather than a subdividing particle, and propagation is purely radial with no refraction [17]. This trade-off is acceptable in this context, because the NPR passes suppress the fine details that would distinguish more accurate techniques from a simpler approximation, and allows for less computationally expensive options.

2.2.1 Ripple wavefront function

Each ripple is defined as a radially symmetric parametric sinusoidal wave shaped by a Gaussian envelope. The expanding ring radius calculated using

$$r(t) = v * t + (r0)$$

where v is the propagation speed and $r0$ is the initial radius, derived from the maximum width along the colliding object's Y -axis so the ripples begin at the object's perimeter rather than the centre. The Gaussian envelope

$$G(d_r) = \exp(-d_r^2 / 2\sigma^2)$$

confines the disturbance to a narrow band around the radius, producing defined rings rather than constant oscillations around the disturbance. The width parameter σ controls how many visible crests appear within each ring.

Two decay terms control the ripple's lifespan: spatial decay D_d shrinks ripples as they travel further from the origin, and time decay D_t decreases their amplitude over time regardless of distance. They are modelled as:

$$D_d = \exp(-\alpha_d \cdot |\mathbf{x} - \mathbf{c}|)$$

$$D_t = \exp(-\alpha_t \cdot t)$$

A growth factor $S(t) = 1 - \exp(-\gamma t)$ rapidly increases the amplitude from 0 to the starting value over a fraction of a second when the disturbance first occurs to prevent instant large displacements that would look unnatural.

Putting these elements together, the complete ripple function is:

$$h(x, t) = A \cdot \sin(d_r \cdot \omega) \cdot G(d_r) \cdot D_d(|x - c|) \cdot D_t(t) \cdot S(t)$$

Where the angular frequency $\omega = 2\pi / \lambda$ controls the spacing between wave crests.

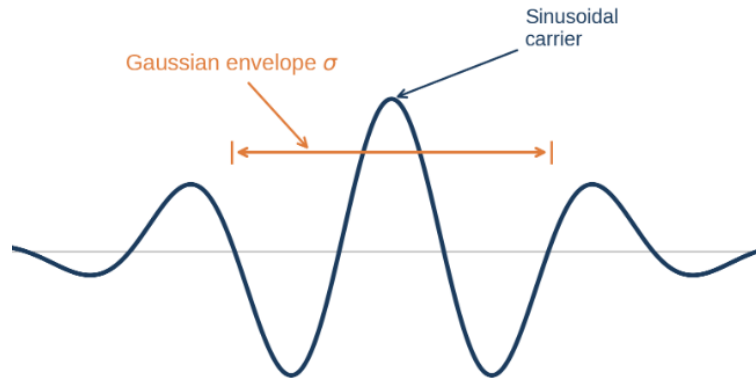


Figure 2.2: Labeled cross section of ripple wavefront

2.2.2 Normals calculation

Normals are calculated by differentiating the displacement function with respect to the surface coordinates, following the same method established for the wave model. The gradient contribution of each ripple is combined with the base wave normal. This method has the same benefits over finite differences as in the wave model.

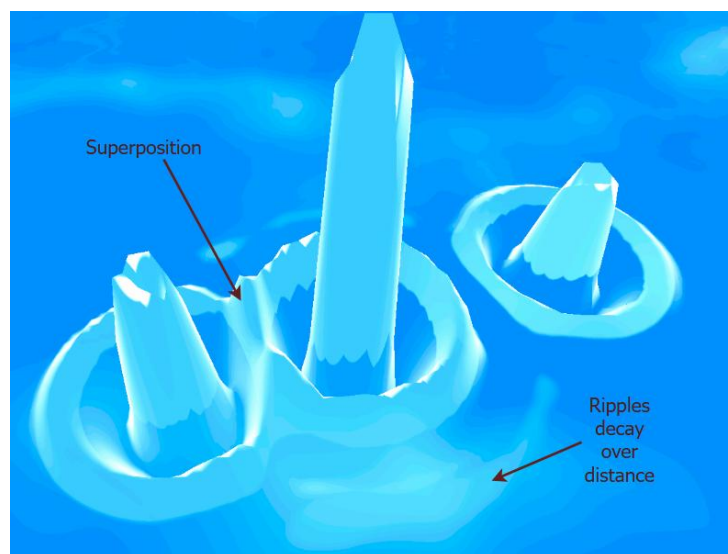


Figure 2.3: Labeled top-down view of overlapping ripples

2.2.3 Ripple management

A fixed circular buffer stores active ripple data on the CPU. On each collision event, the oldest slot is overwritten with the new ripple's origin, radius, amplitude and start time, and the buffer is uploaded to the GPU. The shader iterates over all slots and skips inactive entries via their start time. This design enables constant memory usage and loop length, which is important for predictable performance in complex scenes.

2.2.4 Splash particles

When a disturbance begins, a burst of stretched billboard particles is emitted from the contact point to represent splashing water. Each particle is a camera-facing quad with its length scaled proportionately to its velocity and its opacity scaled over its lifetime. The initial particle velocity is determined by the colliding object's mass so heavier objects result in larger particles that travel further. This approach avoids the cost of volumetric splash geometry and provides full artistic control over the splash particles' appearances.

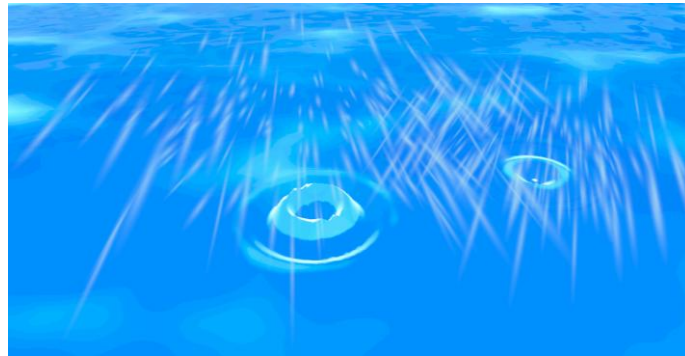


Figure 2.4: Screenshot of splash particles descending after the collision. Left side shows the result from a heavier object, while right is a lighter object

2.3 Foam rendering

As discussed in section 1.3.2.3 of background research, foam forms on ocean surfaces where the kinetic energy of waves concentrate at peaks. Of the foam techniques surveyed, the Jacobian-based approach is the most physically accurate, but requires horizontal vertex displacement to produce the intersections it detects. As established in Section 2.1.1, the wave simulation in this project does not utilise horizontal displacements, meaning the Jacobian method is not suitable.

A heightmap approach was adopted instead. Fragments whose local height exceed a defined threshold are treated as crest regions and layered with foam, with the foam opacity proportional to the distance above the threshold. This requires only a comparison against the height value for each fragment, making it a cheap technique suitable for this project.

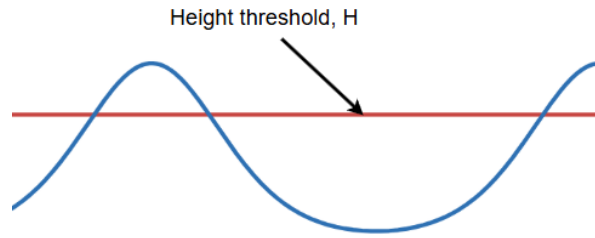


Figure 2.5: Cross section of wave where everything above H is treated as foam

The lack of physical accuracy in this approach is mitigated by the NPR nature of this project. The goal is not to have perfectly realistic physics, so expensive physically-based techniques can be forgone for simpler, cheaper methods.

2.4 Reflections

Looking back to Section 1.3.2.1, reflection techniques range from static cubemaps to hardware ray tracing, each representing a trade-off between accuracy and cost. This project makes use of a hybrid approach: a cubemap skybox provides a base sky reflection, then screen space reflections (SSR) are layered on top to capture the scene geometry. The two layers are blended using a Fresnel term to modulate reflection intensity based on view angle. This layered method follows the approach used by Sea of Thieves, which combines cheap static reflections with screen-space dynamic reflections to balance quality against performance [7].

2.4.1 Cubemap reflections

The base reflection layer utilises a cubemap texture being used as a skybox. The view direction is reflected about the water surface normal, and used to sample this texture to produce the reflections. This was chosen for the sky layer because the sky is largely static within a scene, mitigating the cubemap's inability to capture dynamic objects. Using it as a base layer under the SSR also ensures the surface is never left without any reflections.



Figure 2.6: Screenshot of skybox reflections on the surface at a grazing angle

2.4.2 Screen-Space Reflections

SSR was selected over planar reflections and hardware ray tracing to capture dynamic scene geometry. Planar reflections require an additional screen render per frame, roughly doubling the computational cost, while hardware ray tracing requires dedicated RT cores that weaker hardware may not have. SSR operates within the existing scene buffers, meaning no additional scene renders are needed, making it the most practical option for reflecting dynamic geometry with the project's performance constraints.

2.4.2.1 Ray computation

The reflection rays are computed entirely in view space. The fragment's world-space position p_{ws} and surface normal n_{ws} , along with the view direction v_{ws} are transformed into view space using the view matrix V :

$$p_{vs} = V \cdot p_{ws}$$

$$n_{vs} = V_{\{3 \times 3\}} \cdot n_{ws}$$

$$v_{vs} = V_{\{3 \times 3\}} \cdot v_{ws}$$

Where $V_{3 \times 3}$ represents the upper-left 3x3 submatrix of V , used to avoid translational offsets. The reflection direction is then computed using the standard reflection formula:

$$r_{vs} = v_{vs} - 2(v_{vs} \cdot n_{vs}) \cdot n_{vs}$$

The resulting r_{vs} is the direction the ray will be marched through view space.

2.4.2.2 Ray marching

The ray progresses through view space in discrete steps. At the start of each iteration i , the ray position is updated as:

$$p_i = p_{\{i-1\}} + r_{vs} \cdot s_i$$

Where s_i is the step size at iteration i . The original step size s_0 is a configurable parameter that is incremented by a scaling term to produce progressively larger steps as the ray moves deeper into the scene:

$$s_i = s_{\{i-1\}} + s_0 \cdot (\sigma - 1)$$

where σ is the scaling factor. This acceleration allows the rays to collide with distant geometry without giving up the accuracy provided by smaller step sizes on nearby objects where it matters most.

At each step, the view-space position p_{vs} must be converted into screen UV coordinates to sample the depth buffer. This is done by utilising the projection matrix:

$$p_{clip} = P \cdot (p_i, 1)$$

The clip-space coordinates are then converted to normalised device coordinates (NDC) by the perspective divide, and remapped to UV space:

$$UV = \frac{p_{\{clip\}} \cdot xy}{p_{\{clip\}} \cdot w} \cdot 0.5 + 0.5$$

If $p_{\{clip.w\}} \leq 0$, the ray has passed behind the camera and the ray is terminated.

2.4.2.3 Hit detection

At each step, the scene depth at the converted UV is sampled from the depth buffer and converted to a linear eye-space distance z_scene using the inverse Z-buffer linearisation.

The ray's own eye-space depth is calculated from its view-space position as:

$$z_{ray} = -p_{\{i\}} \cdot z$$

The negation is required in a right-handed coordinate system because objects in front of the camera in view space have negative z values, while eye space is conventionally positive. A hit is detected when the ray has passed behind the scene surface and the penetration depth is within a thickness tolerance T :

$$hit\ condition: z_{ray} \geq z_{scene}\ AND\ |z_{ray} - z_{scene}| \leq T$$

The first condition detects that the ray has crossed behind a surface; the second ensures it has not passed too far through, which would indicate the ray is behind an object rather than hitting its front face. Without the thickness test, rays passing through behind geometry would result in false hits.

This is a linear view-space marching approach, sometimes referred to as the "3D march", in which the ray is stepped in three dimensions and reprojected each iteration, as opposed to the more efficient DDA-based screen-space traversal proposed by McGuire and Mara which steps pixel-by-pixel along the projected 2D ray [11]. The DDA approach was deemed overly complex for this project where the focus is on combining techniques to produce a desired result rather than implementing reflections alone.

2.4.2.4 Binary search refinement

The standard marching iterations above locate an approximate intersection, but it may not be precisely where the geometry is located as the ray can overstep. A bisection phase is needed to refine the result without increasing the step count. When the collision is first detected, on every subsequent iteration, the step direction is reversed, and the step size is halved:

$$s_i = -s_{\{i-1\}} \cdot 0.5$$

This is equivalent to a binary search along the ray, converging towards the true intersection by narrowing down the selection from both sides.

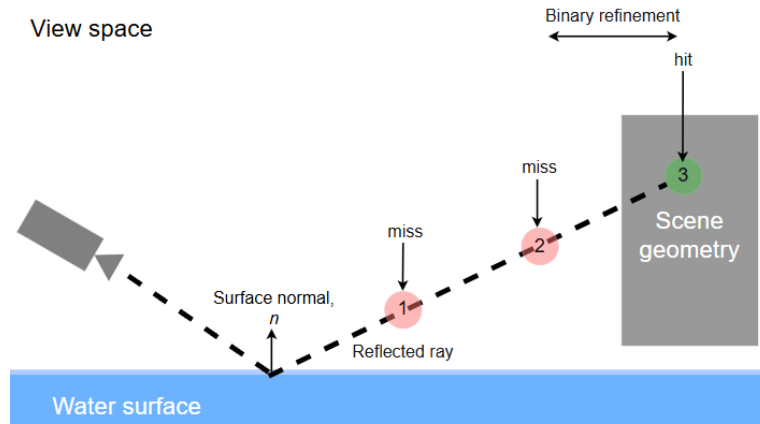


Figure 2.7: Diagram showing the ray march process

2.4.2.5 Boundary and validity checks:

Several conditions cause the ray to be rejected without producing a reflection:

- If UV falls outside the $[0, 1]$ range, the ray has left the visible screen area and is terminated.
- If $p_{clip} \cdot w \leq 0$, the ray has passed behind the camera and is discarded.
- A maximum distance threshold rejects hits on distant geometry where depth buffer precision is unreliable.
- The world-space position of each candidate hit is reconstructed and tested against the water surface height y_{water} . If the hit lies below the water plane, it is rejected to prevent the surface from reflecting itself or underwater geometry. The reconstruction is performed by inverting the projection and view transforms from the hit UV and sampled depth back to world space.

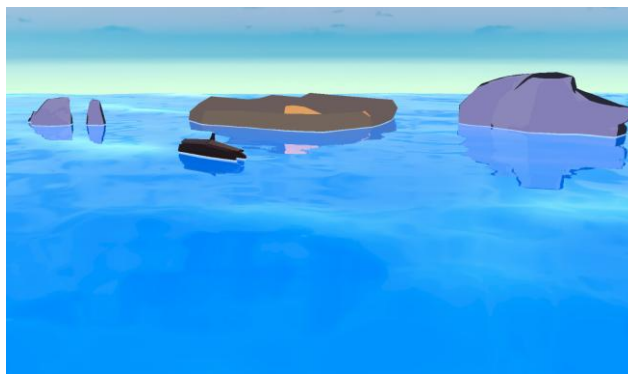


Figure 2.8: A screenshot of the SSR blending with the skybox reflections

2.4.3 Fresnel blending

The reflection intensity is modulated by view angle using the Schlick approximation as discussed in Section 1.3.2.1 [8]:

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos \theta)^5$$

where θ is the angle between the view direction and the surface normal. F_0 is set to 0.02, corresponding to water's refractive index of approximately 1.33. The Fresnel term blends between the scene colour and the combined reflection from the SSR and cubemap layers, so the reflections are more visible at grazing angles, and faint at steep angles.

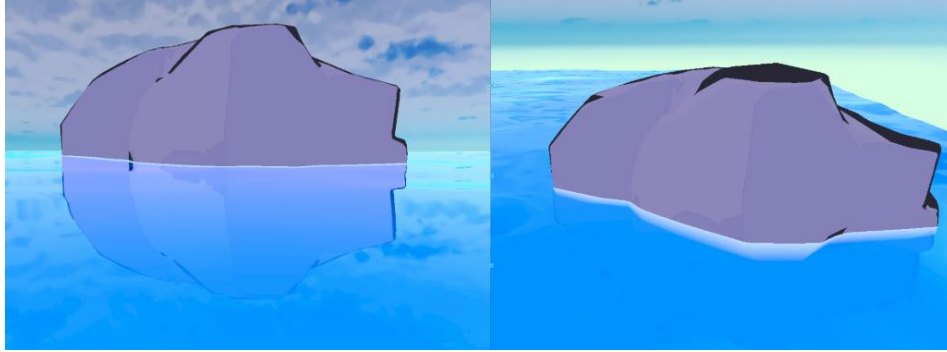


Figure 2.9: Two screenshots demonstrating the Fresnel effect: left is at a grazing angle while right is from a steeper viewing angle

2.4.4 Distance Fade

To address the visual artifacts that SSR produces on geometry from the camera, a distance-based fade is applied to the reflections. The fade is computed from the vertical screen-space distance between the original fragment and the hit UV:

$$fade = 1 - smoothstep(0, d_{offset}, |uv_{hit}.y - uv_{origin}.y|)$$

Where d_{offset} is a configurable fade distance. This suppresses the significant SSR artifacts – false hits on distant geometry and missing reflections on screen edges – at the cost of fading out some correct reflections at long distances.

2.5 Organisation

2.5.1 Gantt Chart

A Gantt chart was used to define the scope and timeline of the project. The initial version of the chart was produced during the first semester alongside the background research phase and was revised iteratively over the course of the project to accommodate unexpected changes in scope and development setbacks. Maintaining a chart provided a clear visual reference for what had been completed, what remained, and how much time was available for each outstanding task.

Each component of the rendering system — wave simulation, dynamic interactions, foam, reflections, and stylisation — was treated as an independent task with its own scope and deadline. Within each task, work followed an iterative cycle: an initial prototype was produced, tested against the visual and performance targets, and adjusted before being merged into the main project. This approach shared the core principles of agile

development, in which features are built up incrementally through repeated cycles of prototyping and refinement rather than a single linear implementation pass.

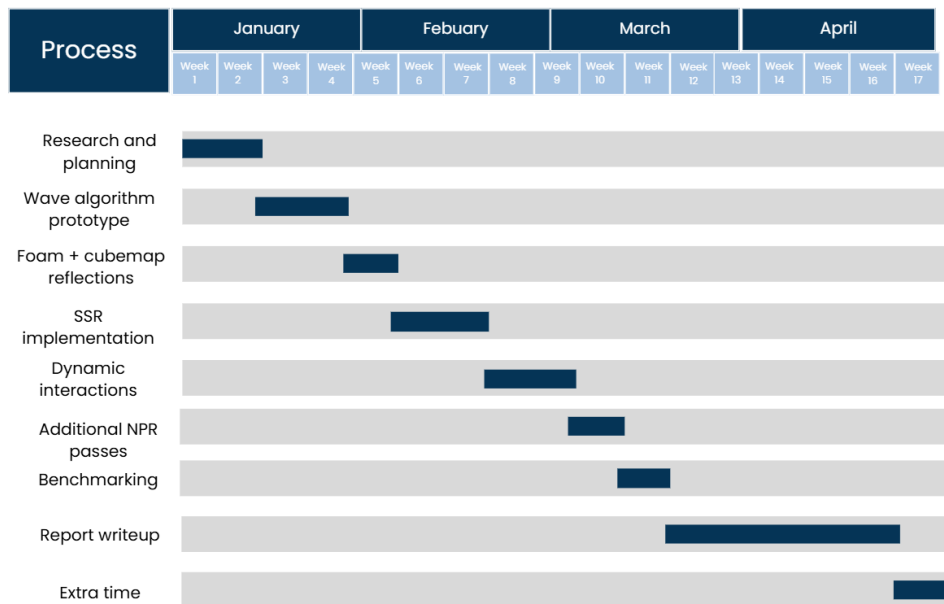


Figure 2.10: Gantt Chart representing the project timeline.

2.5.2 Version control

Git was used throughout the project for source control, with the project hosted on a remote GitHub repository to provide backups and version history. Branches were used to isolate experimental work from the main branch, ensuring that a functioning build of the project was available regardless of the state of any feature in progress. This was particularly important for work that involved multiple systems, where issues could be difficult to diagnose without a working reference to compare against. Notable branches used in development are described below.

- **wave-shaping:** Several shaping functions were prototyped on this branch before the final form was committed to main, allowing comparison of visual output without affecting the main wave system.
- **analytic-normals:** migration of the normal calculation from an initial finite-difference approximation used for prototyping to analytic differentiation within the wave summation loop. This was isolated because the change affected the normal output used by later systems such as reflections, and a faulty normal implementation could have produced subtle issues that would be difficult to attribute to the normal change specifically.
- **ssr-renderer-feature:** SSR implementation required a custom URP render feature to expose copies of the colour and depth buffers to the shader. This branch modified the render pipeline itself rather than shader code, making the risk of breaking changes higher than a typical shader iteration and warranting isolation from main.

Chapter 3

Implementation and Validation

This chapter describes the technical implementation of the stylised water rendering system.

3.1 Chosen tools

Unity was selected as the development environment, making use of the Universal Render Pipeline (URP). URP provides a lightweight rendering pipeline suitable for the project's performance goals, while offering high flexibility in render passes due to its Renderer Feature system. The High Definition Render Pipeline (HDRP) was considered but rejected due to the design being focused on physically-based rendering with higher performance costs. URP's Shader Graph was used to structure the water shader providing a visual interface for linking each main component of the project.

However, shader graph alone was insufficient for the complex subsections. The wave algorithm, normal calculations, ripple evaluation and screen space reflections all required custom HLSL functions that exceeded Shader Graph's built-in capabilities. These were implemented as Custom Function nodes, which allowed raw HLSL to be embedded into the main Shader Graph as standard nodes. This hybrid system maintains the visual simplicity of Shader Graph, while allowing full control over the implementation of core systems.

C# scripts running on the CPU handle dynamic interactions that involve the physics engine such as collision detection and send data to the GPU when needed. They also set up GPU access to the scene's buffers for the main shader.

3.2 Software Architecture

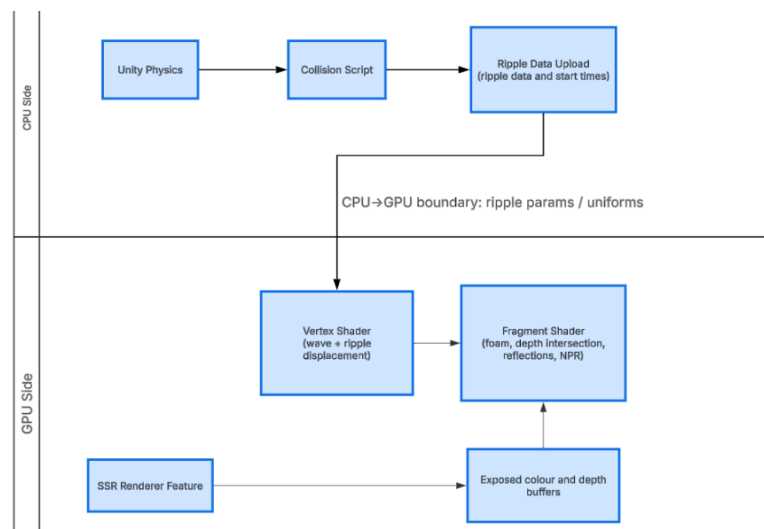


Figure 3.1: High-level system architecture diagram

The system's architecture can be divided into two main sections: vertex-stage operations that define the water surface geometry, and fragment-stage operations that determine the final pixel colour.

3.2.1 Wave simulation

The wave displacement function, described in the methodology as a sum-of-sines model with $\exp(\sin())$ shaping, fBm layering, and domain warping, is implemented as a single HLSL function invoked by a Custom Function node in the vertex stage of the Shader Graph.

3.2.2 Vertex stage

The wave model and ripple system are both implemented as Custom Function nodes executed in the vertex stage of the Shader Graph. Each receives the vertex's world-space XZ position and the current time, along with the shape parameters, and outputs a displaced Y value and a surface normal.

The wave function iterates through the fBm stack in a single loop. Within each iteration the shaped sine value is added to the height and the analytic partial derivatives $\partial h / \partial x$ and $\partial h / \partial z$ are calculated in parallel to avoid unnecessary iterations. At the end of the loop the normal is constructed as $normalize(-\partial h / \partial x, 1, -\partial h / \partial z)$ and returned alongside the height.

The ripple data is stored in an array with size decided at runtime. The ripple function then iterates through this array each frame, with unused elements (identified with a start time of 0) being skipped via an early conditional to prevent wasted computation. For active slots their Gaussian-enveloped sinusoid and analytical normals are computed and outputted.

Both these custom functions are used as inputs to an Add node that combines their heights and normals to be used in the rest of the shader.

3.2.3 CPU – GPU Communication

The ripple system's CPU side is a single C# MonoBehaviour attached to the water plane. It maintains a fixed-size array of ripple data entries managed as a circular buffer. When Unity's *OnTriggerEnter()* callback fires (indicating a Rigidbody has entered the water's trigger collider), the script writes the impact position, object radius, current time, and a mass-scaled amplitude into the next slot and advances the circular index.

Each frame, the ripple data is uploaded to the GPU using *Material.SetVectorArray()* and the start times using *Material.SetFloatArray()*. This transfers the data as uniform arrays that the vertex shader's ripple function can access directly. The upload occurs unconditionally every frame rather than only when a new ripple is added as the GPU needs the updated time deltas for all active ripples regardless.

3.2.4 SSR renderer feature

SSR requires access to the current frame's information, but Unity does not provide this data by default. Use of the deferred render pipeline is needed to create buffers containing the scene surface data. A separate C# script then exposes the colour and normal buffers to the GPU as global 2D textures. This script runs between the opaques and transparent rendering stages to prevent unwanted reflections such as from the skybox.

The SSR itself is implemented as a custom HLSL function that performs the ray march described in section 2.4.2. The function outputs the updated fragment colours along with a mask that indicates where ray collisions occurred along the surface. The output is then linearly interpolated with the skybox reflections based on the mask.

3.2.5 Fragment stage

The remaining rendering operations are linked and combined within the Shader Graph's fragment stage.

1. **Sky reflection.** The normal outputted by the ripple function is reflected across the negated view direction and used to sample the scene cubemap. A Schlick-style Fresnel base, $F = 1 - N \cdot V$, is computed once and reused later by the SSR composite [8]. Here it is raised to *_Cubemap_Fresnel_Strength* and used to blend *_Deep_Color* toward the sky sample, biasing the reflection toward grazing angles.
2. **Screen-space reflections.** Once the SSR function finished running, the returned colour is posterized to reduce the number of colours in the final reflections. The Fresnel approximation F is reused with a separate exponent *_SSR_Fresnel_Strength* allowing the two reflections to have different opacities at various view angles.
3. **Height masks.** Two height-driven masks are computed in parallel. The first is for the surface foam, using a vertical fade factor by clamping $(h_{wave} - \textit{_Sea_Level}) * \textit{_Fade_Multiplier}$ to $[0, 1]$, calculating how far the surface sits above sea level to produce smooth white crests along the wave peaks. The second mask is for the ripples, combining a deterministic Voronoi pattern with the vertical fade factor to produce whitecaps on the ripple peaks that break apart and cluster, mirroring real breaking water.
4. **Object foam.** A separate foam contribution is derived from screen-space depth. The scene depth at the fragment's position is compared against the water surface's own depth and the difference is divided by *_Foam_Cutoff* and saturated, producing a mask that intensifies where the surface meets submerged objects. This output is high where the water sits close to opaque scene geometry, generating a foam band along shorelines and around submerged objects.

5. **Toon stylisation.** The shader is passed through a smoothstep node with a tuneable upper bound *_Step_Transition*, producing the discrete light-to-dark transition characteristic of cel-shaded animation. On top of this, a final posterization pass is applied to produce the cel-shaded colour ramps. This result is used as the output for the shader.

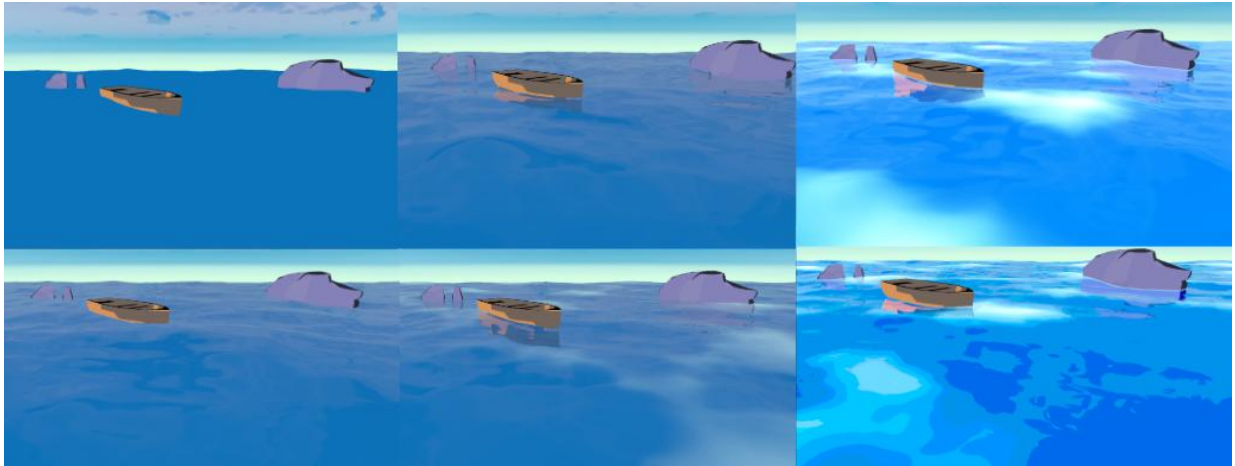


Figure 3.2: Contribution breakdown showing the steps in the fragment shader. From top-left to bottom-right: base colour, cubemap reflections, SSR, foam mask, smoothstep, posterization

3.3 Software validation

Validation during development was carried out using three primary tools provided by the Unity engine: the C# Debug class for script-level logging, the Unity Profiler for measuring per-frame performance, and the Frame Debugger for inspecting each stage of the rendering pipeline.

The C# Debug class was utilized during development of the *ContactDetector* script. Because each collision event drives a ripple spawn, any missed events or duplicate registration would produce visible artefacts in the surface response. *Debug.Log()* was therefore used to log every registered collision alongside its frame index, allowing manual confirmation that events fired on the same frame as the object's intersection with the water plane, and that the recorded collision count matched the number of physical entries.

The Unity Profiler was used to evaluate the performance cost of different reflection techniques during early development. An initial implementation of planar reflections utilised a second camera positioned below the water surface, rendering only opaque geometry to produce a mirrored image. However, profiling revealed that this approach approximately halved the frame rate on an NVIDIA RTX 4070 Super. This overhead was deemed unacceptable for a system targeting lower-end hardware and led to the decision to adopt

screen-space reflections as the primary method for reflecting scene geometry. Further profiling of performance against visual quality is discussed in Section 4.1.1.

The Frame Debugger proved particularly valuable when diagnosing visual artifacts in the screen-space reflection implementation. During development SSR produced severe artifacts at grazing view angles across the water surface. Inspecting the rendering pipeline through the Frame Debugger revealed the cause: the SSR implementation sampled from colour and depth buffers generated after the opaque rendering stage. As the water surface is rendered as a transparent object, it was not yet drawn at the point these buffers were captured. Consequently, rays cast from the water surface were colliding with opaque geometry situated beneath the water plane, producing incorrect reflection samples.

This was an issue specific to displaced transparent surfaces and so did not surface in most SSR implementations, which assume flat opaque surfaces. However, on a moving surface like water, differences in height at different points created this unique bug. Identifying this issue through the frame debugger enabled the decision to take in the fragment height in the SSR function and terminate any rays that pass below this threshold, resolving the artifacts and improving performance.

3.4 GUI

The user may wish to adjust the appearance of the water surface themselves to fit their needs, so a set of customisable parameters were included to provide this. The project makes use of Shader Graph's categories to organise and present them.

3.4.1 Shader Parameters

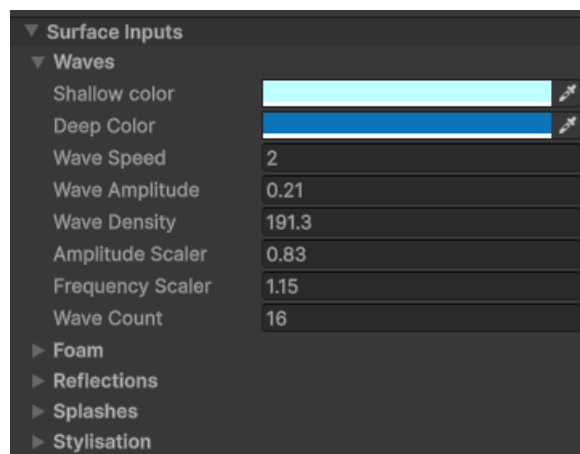


Figure 3.3: Customisable parameter list for the ocean shader

The parameters are grouped by subsystem, allowing the user to adjust and validate each component individually.

Chapter 4

Results, Evaluation and Discussion

4.1 Evaluation

This project was evaluated under two main categories based on the system requirements determined at the start of development: performance and how well it creates 2D animated visuals.

4.1.1 Performance

To evaluate the system's compliance with the real-time performance requirement, frame time measurements were captured across a series of controlled parameter tests. Each test isolates a single configurable parameter (wave count, SSR ray march step count, or maximum concurrent ripple count) while holding the remaining parameters at their default values of 16 waves, 64 SSR steps, and 64 ripple slots. All measurements were recorded at 1920×1080 resolution with VSync disabled on a mid-range consumer system (Intel i5-9400F, NVIDIA GTX 1660 Ti, 16 GB DDR4-2400MHz). Each configuration was sampled over 1500 frames with a fixed camera position and angle to ensure consistency. The test scene contained only the water surface and, for ripple measurements, a scripted object generating impacts at a fixed rate. Frame time in milliseconds is used as the primary metric throughout, as it scales linearly with workload unlike frames per second. On average, 95th percentile (P95), and 99th percentile (P99) values are reported for both average frame time and GPU time to characterise both typical and worst-case performance.

4.1.1.1 Wave Count

Figure 4.1 shows frame time as a function of the number of waves in the fBm model, varied from 8 to 48. The relationship is approximately linear, rising from 6.42 ms at 8 waves to 7.80 ms at 48; an increase of 1.39 ms across a sixfold increase in wave count, or roughly 0.035 ms per wave. This is consistent with the implementation, in which each wave adds a single iteration to the vertex shader's loop. The per-wave cost is small because the operations are arithmetically simple and the loop body contains no texture fetches or branching.

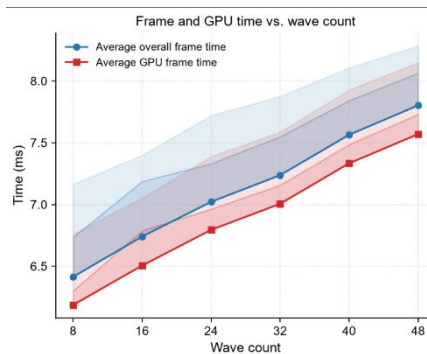


Figure 4.1: Average frame time against wave count. Configuration: 64 SSR steps, 64 ripple slots

The workload is almost entirely GPU-bound. At 48 waves the average GPU time is 7.57 ms against a total frame time of 7.80 ms leaving only 0.23 ms attributable to CPU-side work. The P95 and P99 values track closely with the average throughout the sweep (P99 exceeds the average by no more than 0.5 ms at any point), indicating stable frame pacing with no significant outlier frames. Even at the maximum tested wave count of 48, the system maintains an average frame rate of 128 fps, comfortably above the 60fps standard expected for real-time rendering.

4.1.1.2 SSR step count

This parameter directly controls the maximum number of iterations in the fragment shader's ray-march loop: each step requires a matrix projection from view-space to screen UV, a depth buffer sample, and a comparison against the ray's current depth. Increasing the step count allows smaller step distances to be used, thus increasing the precision of the reflections and preventing rays missing on thin geometry.

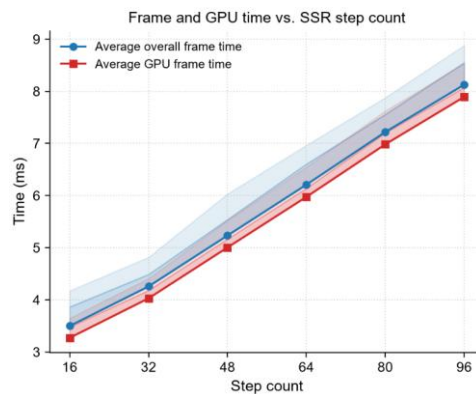


Figure 4.2: Average frame time against SSR step count. Configuration: 16 waves, 64 ripple slots

The scaling is approximately linear throughout, with the average frame time increasing from 3.50 ms to 8.13 ms — a cost of roughly 0.058 ms per additional step. As with the wave count tests, the workload is GPU-dominated. At 64 steps the SSR pass contributes approximately 5.98 ms of GPU time to a total frame time of 6.21 ms, making it the single most expensive component of the rendering pipeline at its default configuration. Despite this, even at 96 steps the system remains above the 60 fps real-time threshold on the test hardware.

4.1.1.3 Ripple Count

Unlike the previous two tests the ripple system spans both the CPU and GPU. The vertex shader evaluates the vertex displacement for each active ripple, while the CPU-side C# script manages the circular buffer of ripple parameters and uploads them to the GPU as uniform arrays each frame.

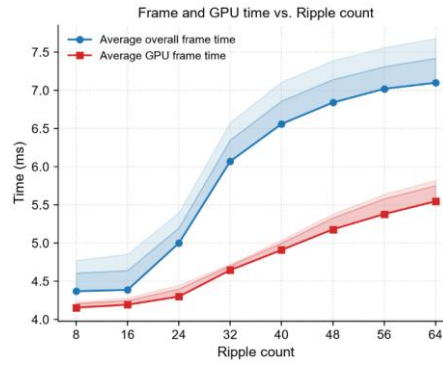


Figure 4.3: Average frame time against Ripple Count Configuration: 16 waves, 64 SSR steps

The GPU cost scales modestly, rising from 4.16 ms at 8 ripples to 5.55 ms at 64 — an increase of 1.39 ms. This is expected as each additional ripple slot adds one iteration to the vertex shader's evaluation loop, each consisting of a radial distance calculation, the Gaussian envelope, and the analytic normal contribution.

It is clear that while the GPU time's P95 and P99 values are controlled and show little difference from the average, the overall frame time's P95 and P99 are noticeably higher than the average. This indicates the system has occasional spikes in CPU performance, which increase as the ripple count increases, going from a 9.2% increase at 8 slots to 22% at 64 slots. This is likely caused by sending increasingly large amounts of data to the GPU each frame, introducing intermittent stalls that affect the worst-case frame time. However, in practice, 64 simultaneous ripples are most likely excessive for real-time scenarios, with 8-16 being more likely, with a much smaller CPU overhead.

4.1.2 Visual Analysis

This section addresses the remaining requirements through a structured visual assessment: Requirement 1 (convincing ocean surface motion), Requirement 2 (real-time dynamic surface interactions), and Requirement 3 (non-photorealistic visual style based on 2D animation conventions). Since motion is inherently difficult to assess from static figures, it is encouraged to view the demonstration video to see the system in motion. The analysis here focuses on surface appearance and interaction quality as captured in representative frames.

Non-photorealistic rendering is, by its nature, a series of stylistic choices rather than a single target to be matched [19]. Evaluating Requirement 3 is therefore done by identifying the principal areas where NPR water rendering differs from physically-based water, situating prior stylised water rendering and 2D animation within that space, and assessing whether the system makes coherent choices in those areas. Figure 4.4 presents the system's output (top left) alongside three reference points spanning the design space: a physically-based real-time FFT ocean [24] representing the photorealistic reference from which NPR water differentiates itself; the cel-shaded real-time water of *The Legend of Zelda: The Wind Waker*

(bottom left); and the hand-drawn animation of *Ponyo* (bottom right). The system's choices are evaluated as positions in these areas alongside the figure.



Figure 4.4: Top left: current system, Top right: physically-based ocean shader, Bottom left: The Legend of Zelda: The Wind Waker, Bottom right: Ponyo

1. **Colour banding.** The surface exhibits discrete regions of flat colour separated by hard boundaries, rather than the continuous smooth gradation characteristic of physically-based ocean shaders like the FFT ocean. This style is mirrored in the Wind Waker and Ponyo images and is a clear departure from the physically-based reference.
2. **Reflection visuals.** While not shown in Figure 4.4, physically-based water exhibits reflections that perfectly mirror geometry and fade rapidly based on a Fresnel term. NPR treatments vary from the clear, simplified reflections seen in Ponyo to omitting them entirely like in Wind Waker. This system utilises colour banding to produce results similar to these simplified reflections with an intentionally weaker Fresnel term to amplify their role in the simulation.
3. **Surface detail.** The FFT ocean shows high-frequency details with ripples and choppy waves. NPR treatments reduce this to a small number of defined waveforms, as in Wind Waker, or even no movement at all, as in Ponyo. The system sits toward the simplified end of this area but can shift to a more realistic appearance by reducing the posterization and increasing the wave count, making it highly flexible.
4. **Foam simplification.** The FFT ocean shader generates foam from waves breaking and overlapping, producing dense and chaotic patterns. NPR treatments use foam more selectively: as decorative motif (Wind Waker's swirl patterns) or to signify crests on the water surface (Ponyo). This system replicates this approach, restricting foam to wave crests and contact points with object.

5. **Lighting detail.** The lighting is consistent throughout the surface, avoiding the high-frequency shimmering seen in physically realistic renderers that would break the animated quality of the surface. This flat lighting is seen in both *Wind Waker* and *Ponyo*, as the complex lighting would compete with the cel-shaded colours.

Put together, these properties demonstrate that the system produces a visual style that is identifiably non-photorealistic and broadly consistent with the conventions of 2D animated water satisfying Requirement 3. However, two notable omissions should be acknowledged:

1. **Edge outlines.** The system does not generate visible edge outlines on the water surface or along wave crests, a technique used in some 2D animation styles such as *Ponyo* to emphasise surface contours and silhouettes. This may limit the wave readability at low amplitudes and the possible styles that can be recreated.
2. **Specular highlights.** While the system deliberately omits the broad-spectrum specular shimmer characteristic of physically-based water, it also omits the stylised sun-glint that some 2D animation employs as a discrete visual element. Adding this would be consistent with the cel-shaded treatment elsewhere in the system.

Requirement 1 asks that the simulation visibly emulate the movement of water in a deep ocean environment. The wave surface in Figure 4.4 exhibits visible height variation across the plane, with broad primary waves overlaid by finer secondary details. The waves also visibly influence each other, and do not look repetitive at low elevations. However, when looking at the plane from high above the tiling nature of the sum-of-sines algorithm becomes visible. While these properties are evident in the static frame, the full contribution of the wave algorithm is clear in the demonstration video.

Requirement 2 asks that dynamic surface interactions integrate visually with the underlying simulation. Figure 4.4 shows a ripple expanding outward from an impact point. This is evaluated in the same vertex stage as the wave displacement and summed into the same height field, meaning the ripples exist on top of the existing wave motion rather than overwriting it. The splash particles are emitted at the moment of collision and decay over their lifetime, providing a stylised representation of ocean spray dispersing in the air. Combining these elements, the interaction system produces a visually coherent response that does not appear disconnected from the rest of the surface, satisfying Requirement 2.

4.2 Conclusions

This project set out to develop a real-time stylised water rendering system that combines ocean simulation techniques with non-photorealistic rendering to produce a visual style consistent with 2D animation conventions. The resulting system implements sum-of-sines

wave displacement with fBm layering and domain warping, heightmap-based foam, screen-space and cubemap reflections with Fresnel-driven blending, and a dynamic interaction system containing analytic ripple propagation and splash particle emission — all integrated within a single custom shader pipeline running in real time. As demonstrated in the previous section, the system satisfies all four requirements defined at the outset of the project: convincing ocean surface motion, a non-photorealistic visual style inspired by 2D animation conventions, and dynamic surface interactions that integrate visually with the underlying simulation, that all run in real time on mid-range hardware.

However, the system is not without limitations. As mentioned previously, techniques used in some stylised works such as outlines and specular highlights have not been implemented. There are also optimisation concerns that limit the system's scalability such as the CPU bottleneck at high ripple counts. Yet, these limitations represent clear avenues for further development rather than fundamental shortcomings of the system, and none prevent the system from meeting its stated requirements

Overall, the project demonstrates that NPR techniques can be utilised to produce high-quality ocean rendering with real-time dynamic interactions without requiring the computational overhead of physically accurate techniques.

4.3 Future work

There are several areas for potential future work with this project:

The main route further development could take is adding other techniques to allow a wider range of ocean shapes and styles to be represented.

1. **FFT wave simulation.** FFT would allow for more larger waves that would enable larger areas to be covered without tiling.
2. **Further interactions.** Dynamic interactions such as wakes and trails behind moving objects would create a more responsive surface.
3. **NPR additions.** techniques such as outline detection could be implemented increase the number of art styles the system can achieve.

Additionally, further optimisation could be done on the system's performance, for example on the ripple management system. As shown in section 4.1.1.3, the CPU-side logic introduces occasional frame time spikes at high ripples. This could be resolved by implementing the system as a compute shader on the GPU, allowing the ripple data to be managed by the GPU entirely rather than constantly fetching it from the CPU.

Finally, a user survey could be a strong addition to the evaluation section for this project to gather opinions from people of different backgrounds to assess the success of the system in achieving its visual requirements.

List of References

- [1] Emmanuelle Darles, Benoît Crespin, Djamchid Ghazanfarpour, and Jean-Christophe Gonzato. A survey of ocean simulation and rendering techniques in computer graphics. *Computer Graphics Forum*, 30(1):43–60, 2011.
- [2] Darwyn R. Peachey. Modelling waves and surf. *ACM SIGGRAPH Computer Graphics*, 20(4):65–74, 1986.
- [3] Mark Finch. Effective water simulation from physical models. In Randima Fernando, editor, *GPU Gems: Programming Techniques, Tips, and Tricks for Real-Time Graphics*, chapter 1. Addison-Wesley, 2004.
- [4] Alain Fournier and William T. Reeves. A simple model of ocean waves. *ACM SIGGRAPH Computer Graphics*, 20(4):75–84, 1986.
- [5] Inigo Quilez. Domain warping. <https://iquilezles.org/articles/warp/>. Accessed: April 2025.
- [6] Jerry Tessendorf. Simulating ocean water. In *SIGGRAPH 2002 Course Notes #9 (Simulating Nature: Realistic and Interactive Techniques)*. ACM Press, 2002.
- [7] Nigel Ang, Andy Catling, Francesco Cifariello Ciardi, and Valentine Kozin. The technical art of sea of thieves. In *ACM SIGGRAPH 2018 Talks*, pages 1–2, 2018.
- [8] Christophe Schlick. An inexpensive BRDF model for physically-based rendering. *Computer Graphics Forum*, 13(3):233–246, 1994.
- [9] Ned Greene. Environment mapping and other applications of world projections. *IEEE Computer Graphics and Applications*, 6(11):21–29, 1986.
- [10] Unity Technologies. Reflection probes. <https://docs.unity3d.com/6000.3/Documentation/Manual/ReflectionProbes.html>. Accessed: April 2025.
- [11] Morgan McGuire and Michael Mara. Efficient GPU screen-space ray tracing. *Journal of Computer Graphics Techniques (JCGT)*, 3(4):73–85, 2014.
- [12] Tomas Akenine-Möller, Eric Haines, Naty Hoffman, Angelo Pesce, Michał Iwanicki, and Sébastien Hillaire. *Real-Time Rendering, Fourth Edition*. A K Peters/CRC Press, 2018.
- [13] Jerry Tessendorf. Simulating ocean water. In *SIGGRAPH 2001 Course Notes (Simulating Nature: Realistic and Interactive Techniques)*. ACM Press, 2001.
- [14] Jonathan Dupuy and Eric Bruneton. Real-time animation and rendering of ocean whitecaps. In *SIGGRAPH Asia 2012 Technical Briefs*, Article 15, pages 1–3, 2012.
- [15] Michael Kass and Gavin Miller. Rapid, stable fluid dynamics for computer graphics. *ACM SIGGRAPH Computer Graphics*, 24(4):49–57, 1990.

- [16] Jerry Tessendorf. Interactive water surfaces. In Andrew Kirmse, editor, *Game Programming Gems 4*, pages 265–274. Charles River Media, 2004.
- [17] Cem Yuksel, Donald H. House, and John Keyser. Wave particles. *ACM Transactions on Graphics (Proceedings of SIGGRAPH 2007)*, 26(3), Article 99, 2007.
- [18] James F. O'Brien and Jessica K. Hodgins. Dynamic simulation of splashing fluids. In *Proceedings of Computer Animation '95*, pages 198–205, 1995.
- [19] Jan Eric Kyprianidis, John Collomosse, Tinghui Wang, and Tobias Isenberg. State of the 'art': A taxonomy of artistic stylization techniques for images and video. *IEEE Transactions on Visualization and Computer Graphics*, 19(5):866–885, 2013.
- [20] Adam Lake, Carl Marshall, Mark Harris, and Marc Blackstein. Stylized rendering techniques for scalable real-time 3D animation. In *NPAR 2000: Proceedings of the 1st International Symposium on Non-Photorealistic Animation and Rendering*, pages 13–20, 2000.
- [21] Pascal Barla, Joëlle Thollot, and Lee Markosian. X-toon: An extended toon shader. In *NPAR 2006: Proceedings of the 4th International Symposium on Non-Photorealistic Animation and Rendering*, pages 127–132, 2006.
- [22] Takafumi Saito and Tokiichiro Takahashi. Comprehensible rendering of 3-D shapes. *ACM SIGGRAPH Computer Graphics*, 24(4):197–206, 1990.
- [23] Inigo Quilez. fBm. <https://iquilezles.org/articles/fbm/>. Accessed April 2025.
- [24] Acerola / Garrett Gunnell. Water. <https://github.com/GarrettGunnell/Water>. Accessed: April 2025.
- [25] John Van Ness and Benoit Mandelbrot. Fractional Brownian Motions, Fractional Noises and Applications, pages 422-437, 1968.

Appendix A Self-appraisal

A.1 Critical self-evaluation

Working on this project has been both challenging and rewarding. It pushed me into areas I had little prior experience with, particularly shader programming in HLSL, and gave me the opportunity to do research into NPR, a field I have always been interested in. Looking back, I am proud of what I produced, and I plan to keep developing the renderer after submission and use it in my own game projects in the future.

The project went through a lot of change during the early prototyping phase. My original idea was to render stylised waterfalls, but after some initial experimentation it became clear that the techniques involved did not really align with the wave simulation literature I wanted to draw on. I then moved to rivers, which were closer but still felt restrictive in terms of the range of effects I could showcase. Eventually I settled on ocean scenes, which gave me the breadth I needed to demonstrate wave simulation, foam, ripples, reflections, and stylisation all in one scene. While I am happy with where I ended up, I should have committed to a subject sooner. The shifting target meant that some early implementation work had to be thrown away, and it made it harder to scope the project properly until quite late on.

Scope creep was the biggest issue I faced during development. Because stylised water naturally invites more and more features (different wave types, multiple reflection methods, foam variants, dynamic interactions, caustics, underwater effects) it was hard to draw a clean line on what was essential to the project. This led to the future of the project being unclear at times and getting lost in unnecessary development. With hindsight, I should have written a fixed feature list early on and treated anything beyond it as a possible extension rather than something to attempt during the main implementation.

The other major challenge was implementing screen space reflections. I had little prior experience with HLSL going into the project, and SSR turned out to be unforgiving as a first serious shader to write. Getting the ray marching to behave correctly across different camera angles and wave conditions took a great deal of debugging and experimentation, and there were several points where I considered dropping it in favour of a simpler reflection method. Sticking with it was the right call in the end as it ended up as the part of the renderer I am most pleased with, but it consumed considerably more time than I had budgeted for, forcing me to pass on other potential features.

A.2 Personal reflection and lessons learned

The biggest lesson from this project is the importance of fixing scope early and resisting the temptation to keep adding to it. The freedom to choose my own features was something I really enjoyed, but it also meant I had no external pressure to stop, and I underestimated how much that would matter. In future projects I will set a feature list at the start, write down what is in and what is explicitly out, and treat that document as something to argue against rather than something to quietly extend. This solution also helps with my issue of repeatedly changing my project early into development, as I will settle on one strong idea clearly mapped out early.

Learning HLSL while implementing SSR was difficult, but it was also probably the most valuable technical experience I had during the project. It forced me to read other people's shader code carefully and to write small debug visualisations to understand what each section of the code was doing. This process of incremental development is something I will take into future shader projects, as they often lack the powerful debugging tools used in CPU-side scripts.

For me, this project confirmed the part of graphics work I find the most engaging is when complex algorithms are combined with an artistic target. The process of developing and combining rendering techniques to achieve a specific goal has been compelling for me, and provided a clear idea of what I wish to work on next. Overall, I have learnt a lot throughout the development of this project, and plan to extend it after submission to use in my own games going forward.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

The main legal consideration concerns the licensing of third-party material used during development. The implementation was built on a commercial game engine and uses a small number of external assets — shader utility code, skybox textures, and reference imagery — each of which carries its own licence. I checked the terms for each before incorporating them, credited the original authors, and ensured I was within the scope of its licence. The project does not collect, store, or process any personal data, so data protection legislation such as the UK GDPR is not relevant, and the renderer is not being distributed to end users, so consumer-facing software legislation does not apply at this stage either.

A.3.2 Social issues

The project has limited direct social impact, as it is a self-contained graphic project rather than a commercial system. It does not make decisions about people, mediate communication, or interact with users in any way that could affect them socially, economically, or politically.

A.3.3 Ethical issues

This project did not involve human participants, animals, personal data, or any other subject matter that would require ethical review. The evaluation was carried out by me on my own hardware. There are no dual-use concerns either, as a stylised water renderer has no plausible misuse pathway, and the project does not involve machine learning, generative content, or any other technology where bias or misrepresentation would be a meaningful concern.

A.3.4 Professional issues

The BCS Code of Conduct sets out four duties for projects, each of which I have tried to uphold during this project:

A.3.4.1 Public Interest

Since the project has been developed individually, public interest is not affected. The project does not handle personal data or affect public health, safety, or the environment in any meaningful way.

A.3.4.2 Professional Competence and Integrity

The project required me to learn HLSL and several shader techniques from a low starting point. I approached this by reading primary literature alongside informal sources, testing each component before integrating it, and being open in the report about which parts took longer than expected and why. The limitations of the work are stated honestly in both the evaluation chapter and self-appraisal.

A.3.4.3 Duty to Relevant Authority

As a student, my relevant authority is the University and my supervisor. I attended the scheduled meetings, acted on the feedback I was given, kept to the School's submission requirements and academic integrity rules, and did not misrepresent the state of the project at any point.

A.3.4.4 Duty to the Profession

I have been careful to ensure all sources used in this project are credited so any readers can follow the research behind each technique. Nothing in the project misrepresents results or takes credit for work done by others.

Appendix B

External Materials

Material	Purpose	Link
Unity	Development platform and library that the system was built in.	https://unity.com
Boats- PolyPack	Boat models used for the demo scene in Unity	https://assetstore.unity.com/packages/3d/vehicles/sea/boats-polypack-189866
Environment Pack - Rocks Stylized	Rock models used for the demo scene in Unity	https://assetstore.unity.com/packages/3d/environments/environment-pack-rocks-stylized-316250
Cartoon Shader	Applied to scene geometry in the demo scene to fit with the stylised water	https://assetstore.unity.com/packages/vfx/shaders/cartoon-shader-311082
Draw.io	To create diagrams and figures for the report	https://app.diagrams.net